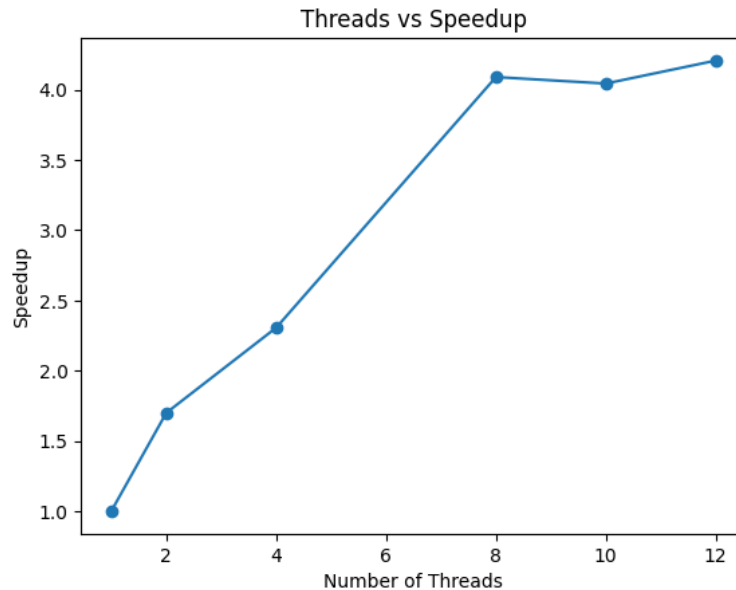


Q1.



The Lennard-Jones force calculation shows clear performance improvement with parallel execution. The execution time decreases from **0.00895 s (1 thread)** to **0.00219 s (8 threads)**, achieving a speedup of about **4.09×**, which indicates effective parallelization. Increasing threads beyond 8 provides only marginal improvement ($\approx 4.21\times$ at 12 threads), showing diminishing returns due to synchronization overhead and memory access contention. The perf statistics show that about **5 CPU cores were effectively utilized**, confirming good parallel workload distribution. The instructions-per-cycle value (~ 1.16 – 1.18) indicates efficient computation, while the high backend-bound percentage ($\sim 45\%$) suggests the processor often waits for memory operations. Overall, the experiment demonstrates good scalability up to moderate thread counts, with performance limited at higher threads by memory bandwidth and cache access delays.

Q2.

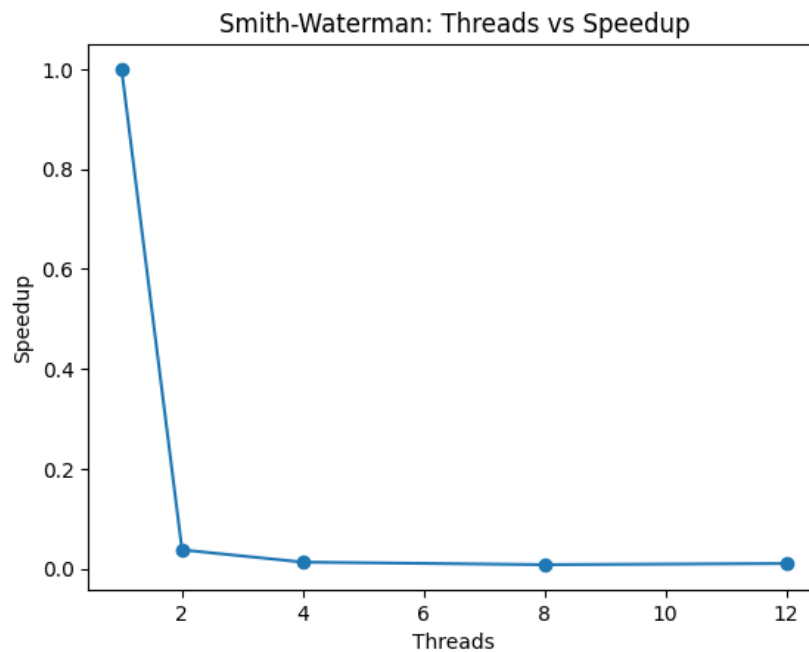
Alignment Score Matrix:

```
0 0 0 0 0 0 0 0 0 0
0 2 1 0 2 1 2 1 2
0 1 1 3 2 4 3 4 3
0 2 1 2 5 4 6 5 6
```

```

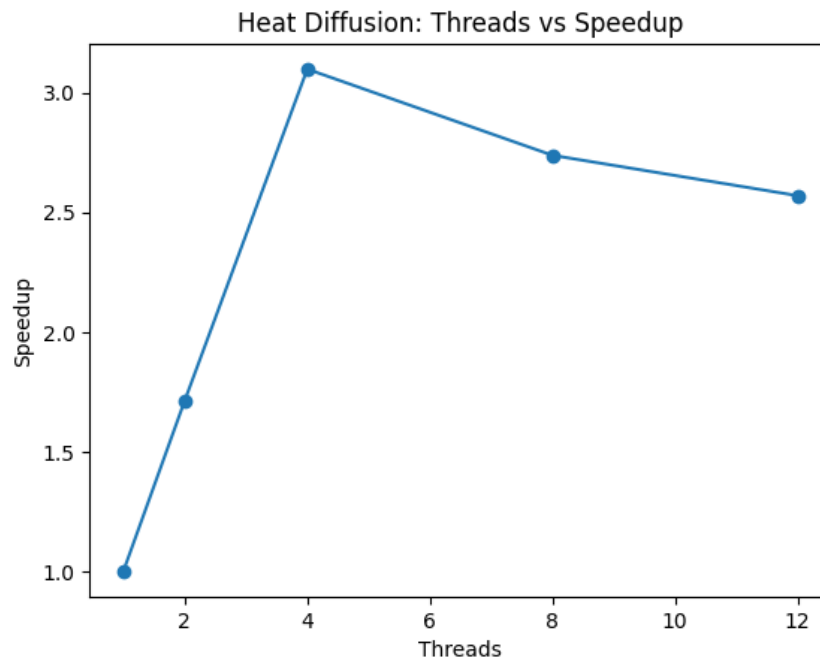
0 1 1 3 4 7 6 8 7
0 2 1 2 5 6 9 8 10
0 1 1 3 4 7 8 11 10
0 0 0 2 3 6 7 10 10
0 2 1 1 4 5 8 9 12

```



The Smith–Waterman experiment shows that parallel execution does not improve performance for this small problem size. The execution time increases from **0.000008 s (1 thread)** to **0.000987 s (8 threads)**, and the speedup drops sharply to **0.008**, indicating parallel slowdown. This occurs because the scoring matrix is very small, so thread creation and synchronization overhead dominate the actual computation. The performance counters show low CPU utilization (~1.6 cores) and a low instructions-per-cycle value (~0.7–0.8), indicating inefficient parallel execution. The backend-bound and frontend-bound percentages suggest the processor spends time waiting on instruction delivery and pipeline inefficiencies rather than performing useful work. Overall, the results demonstrate that dynamic programming algorithms with small input sizes and data dependencies do not benefit from parallelization, and parallel overhead outweighs performance gains.

Q3.



The heat diffusion program shows clear improvement with parallel execution. The execution time reduced from **0.135 s (1 thread)** to **0.0437 s (4 threads)**, achieving a speedup of about **3.1×**, which indicates efficient parallelization and good workload distribution. Beyond 4 threads, performance slightly decreases, with speedup dropping to **2.74× (8 threads)** and **2.57× (12 threads)** due to memory bandwidth limits and increased synchronization overhead. The perf statistics show high CPU utilization (~7 cores used) and efficient execution with about **1.8 instructions per cycle**, indicating effective computation. The backend-bound metric (~24–58%) suggests the processor spends significant time waiting on memory access, confirming a memory bottleneck at higher thread counts. Overall, the results demonstrate good scalability up to moderate thread counts, with performance limited by memory subsystem constraints.